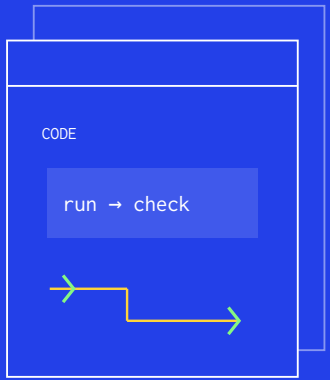
WEEK 1 / LECTURE 01

MLOps: Machine Learning Operations

Foundations of Machine Learning Operations

From experiments to reliable, maintainable ML services.

TRAN Cong Minh



Course agenda

Five weeks · eight lectures · one cumulative ML system.

W1	L1 · MLOps foundations	L2 · Data quality & baselines
W2	L3 · Reproducible environments	L4 · Prediction APIs & testing
W3	L5 · Observable deployment	L6 · Continuous integration
W4	L7 · Training pipelines & gates	L8 · Incident response & rollback
W5	Project clinic · final verification · team presentations	

SECTION 1

What is MLOps?



Machine Learning plus Operations: the vocabulary used throughout the course.

What is MLOps?

MLOps = Machine Learning Operations

ML

Machine Learning

Learn from data to improve performance on a task.

Train and evaluate models that generalize to new data.

+

Ops

Operations

Run ML systems reliably in production.

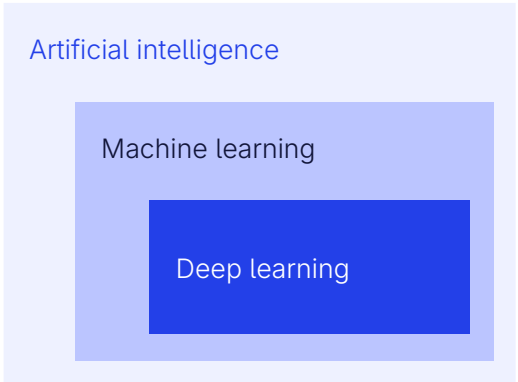
Deploy, serve, monitor, scale, and maintain them over time.

MLOps brings ML development and operations together through automation, testing, deployment, and monitoring.

Adapted from: Mitchell (1997), Machine Learning; Google Cloud Architecture Center, MLOps.

AI vs ML vs DL

Deep learning is a subset of machine learning; machine learning is a subset of AI.



ARTIFICIAL INTELLIGENCE

Ability of a machine to perform cognitive functions we associate with human minds, such as perceiving, reasoning, learning, problem solving, and even creativity

MACHINE LEARNING

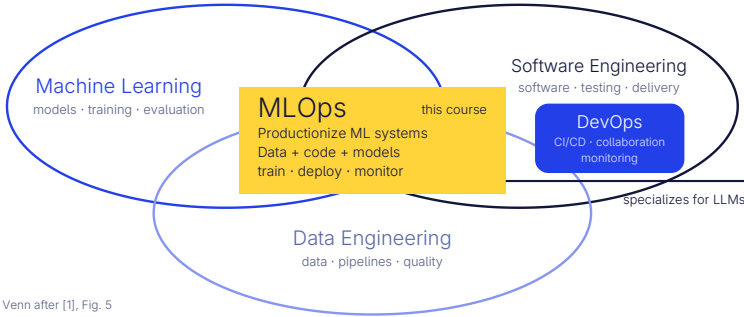
AI techniques that give machines the ability to learn from data without being explicitly programmed, i.e. to automatically improve through experience

DEEP LEARNING

Type of Machine Learning built upon the concept of interconnected layers known as “neurons” that form a neural network.

The Ops landscape

Related operational disciplines share practices, but they do not form a single inheritance tree.



Venn after [1], Fig. 5

LLMOps

LLM systems [4]

Adapt · evaluate
deploy · monitor
prompts · retrieval · models

DataOps

data & analytics [2]

Automate and operate reliable data pipelines
quality · testing · delivery

AIOps

AI for IT operations [3]

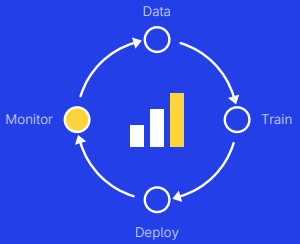
Use AI to detect, diagnose, and automate IT operational issues.
Different direction: AI → IT operations

[1] Kreuzberger et al., IEEE Access 11, 2023, doi:10.1109/ACCESS.2023.3262138.
[2] Mainali et al., DATA ANALYTICS 2021, pp. 61–69.
[3] Diaz-de-Arcaya et al., ACM Comput. Surv. 56(4), 2024, doi:10.1145/3625289.
[4] Pahune & Akhtar, Information 16(2), 87, 2025, doi:10.3390/info16020087.

SECTION 2

Why MLOps?

What turns growing AI capability into a reliable system?



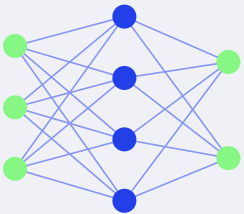
Why now?

Three advances make modern AI practical.



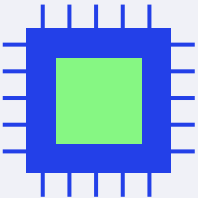
Large Datasets

More observations and richer signals.



Better Models

Stronger learning methods and representations.

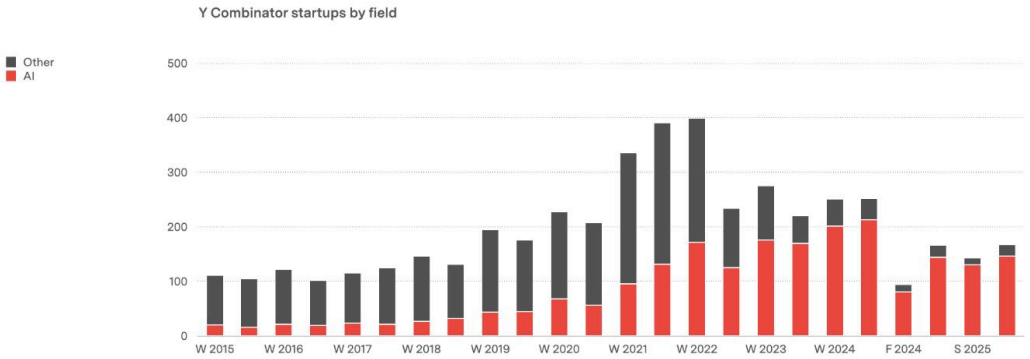


Lots of Computation

Accelerators and scalable training infrastructure.

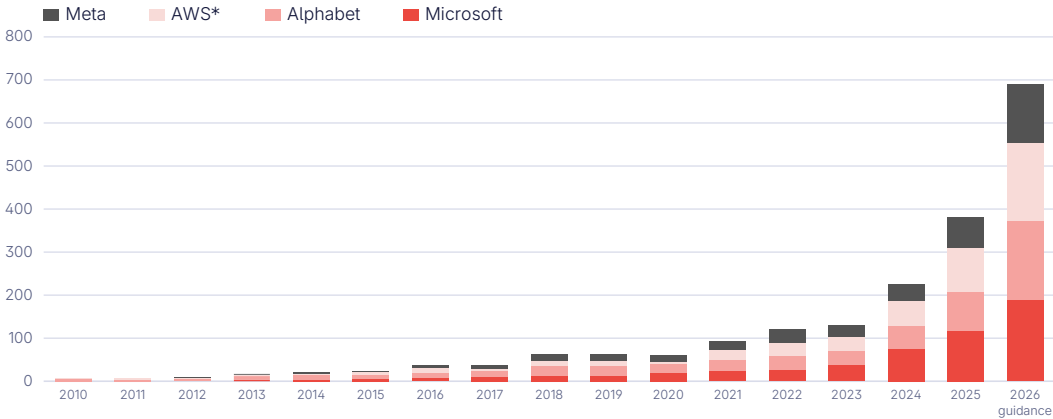
Investment is increasingly concentrated on AI.

Startup mix, not total investment dollars: Y Combinator batches, 2015–2025.



AI demand expands investment in cloud and data centers.

Company capital expenditure (USD bn); 2026 figures reflect company guidance.



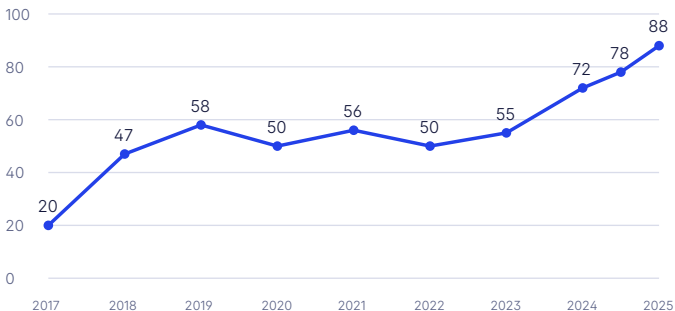
Source: Benedict Evans, "AI eats the world", May 2026; company guidance.
Includes capital leases. *Amazon does not break out AWS capex directly but reports it as the vast majority.

AI adoption has risen sharply in recent years.

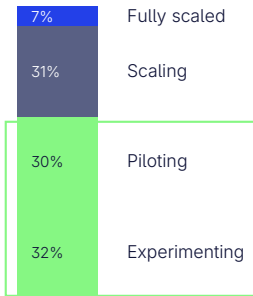
Adoption keeps rising, but most organizations using AI are still experimenting or piloting.

Source: McKinsey Global Surveys on the state of AI, 2017–25.
Share of respondents using AI in at least one business function (definitions changed in 2017–19).

Organizations using AI (%)



Phase of AI use in 2025



62%
experimenting or piloting

Among organizations using AI

A few examples of ML applications.

Different domains, same pattern: a model that becomes a system someone has to run.


Facial recognition


Product recommendation


Email spam filtering


Autocomplete


Finance predictions


Healthcare imaging


Weather forecast


Document AI


Pricing


Video games


Quality control


Robotics


Customer clustering


Customer support

800+

real-world case studies from 150+ companies

Browse the full database: evidentlyai.com/ml-system-design (ML and LLM system design case studies; last updated Dec 22, 2025).

The model is not the system

A successful notebook leaves many engineering responsibilities unresolved.

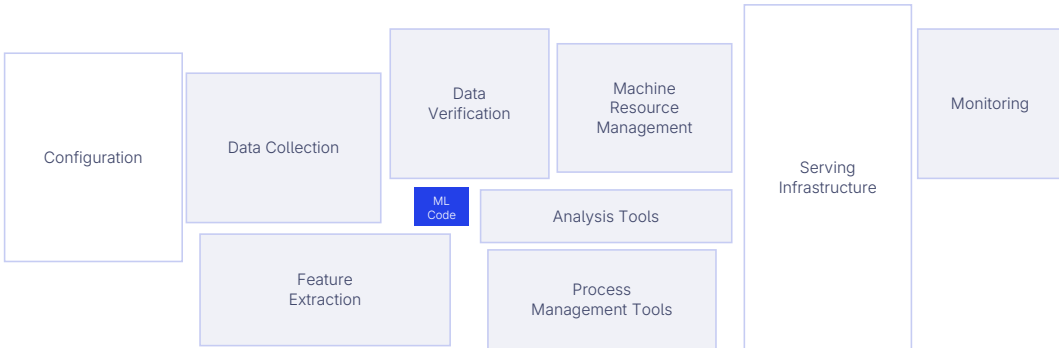


Figure 1. Only a small fraction of real-world ML systems is composed of the ML code, as shown by the small blue box in the middle. The required surrounding infrastructure is vast and complex.

Find the small ML-code block. Which surrounding blocks of your system need tests and ownership?

Redrawn from Sculley et al. (2015), Hidden Technical Debt in Machine Learning Systems, NIPS, Figure 1.

Key definitions

Four terms we will keep using, from the product down to the practice.

THE PRODUCT

ML Application

The final solution or program powered by a Machine Learning model.

THE COMPONENTS

ML System

All the components responsible for the implementation and management of the data and models powering an ML application.

THE ACT

ML Systems Design

The act of designing the architecture and implementing an ML System.

THE PRACTICE

MLOps

A set of practices and tools to enhance and accelerate the entire ML model development life cycle.

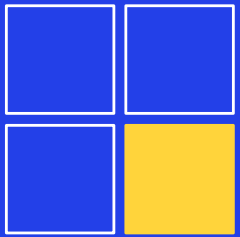
A BRANCH OF MLOPS

LLMOps

Specifically focusing on LLM application development life cycle.

SECTION 3

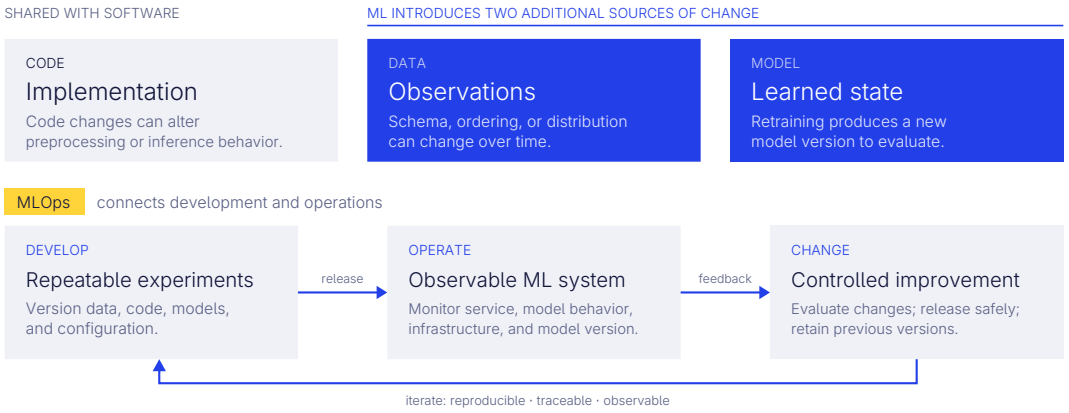
Key concepts of MLOps



The moving parts, the loop and what can go wrong.

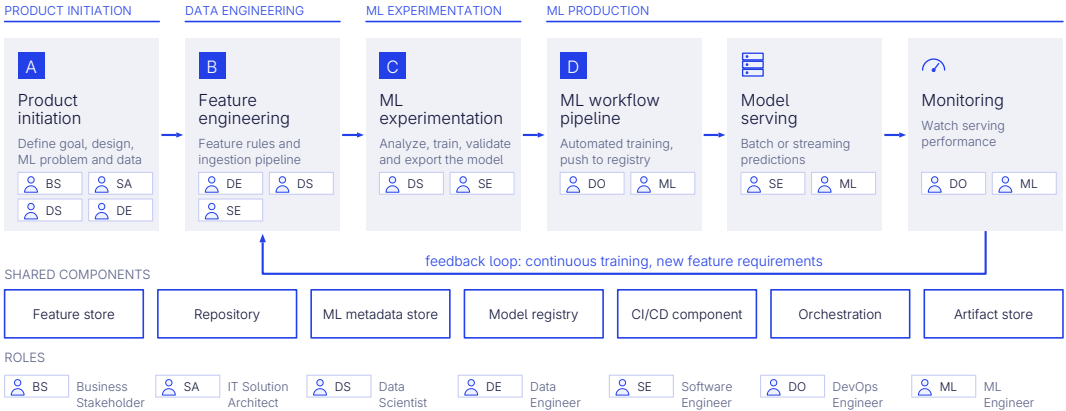
What MLOps adds to ML

ML systems depend on code, data, and learned models; MLOps keeps changes reproducible, traceable, and observable.



End-to-end MLOps overview

Stages, shared components and roles of one MLOps workflow, simplified.



Simplified redraw after Kreuzberger, Kühl & Hirschl (2023), IEEE Access, Fig. 4: end-to-end MLOps architecture and roles.

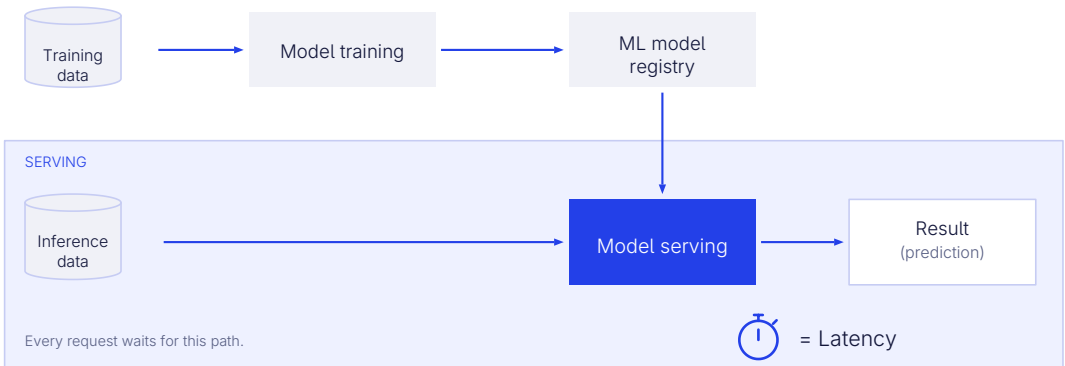
Data preparation

It all starts with data. How to go through all these steps efficiently and effectively.



ML model serving

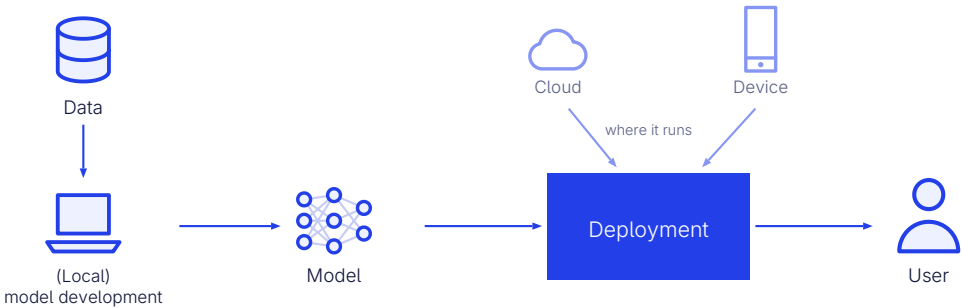
How to efficiently serve an ML model to a client.



Serving answers a request: validate the inputs, then infer with the loaded model.

ML model deployment

How to efficiently deploy your model for serving.



Deployment makes a version available: install the service, then verify what clients actually reach.

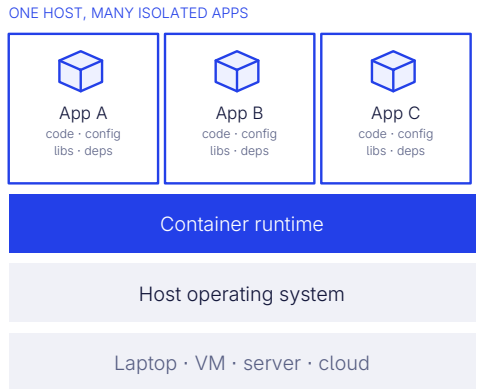
Containerization

Containers encapsulate an application as a single executable package that contains all the information to run it on any hardware:

- Application code
- Configuration files
- Libraries
- Dependencies

Abstracts the application from its host operating system.

Containers move easily from a desktop computer to a VM, or from Linux to Windows, and run consistently on virtualized infrastructure or bare-metal servers, on-premise or in the cloud.

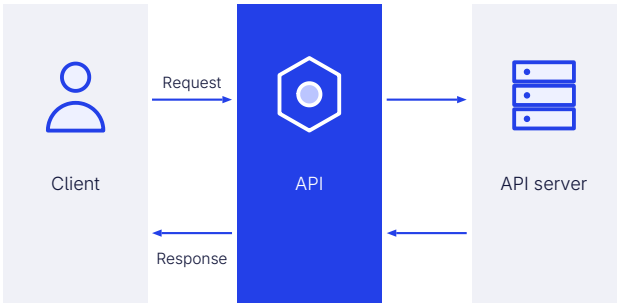


Packaging: Docker bundles code and dependencies; verify its environment and artifact.

Source: IBM, Containerization (ibm.com/topics/containerization).

APIs

Allow other services to call your model or application.



An Application Programming Interface (API) is a set of protocols that enable different software components to communicate and transfer data.

Developers use APIs to bridge small chunks of code into applications that are powerful, resilient and secure, and meet user needs.

Cloud infrastructure

Cloud platforms provide storage, compute, training, deployment and monitoring, and more.



Google Cloud

GCP · Vertex AI



Amazon Web Services

AWS · SageMaker



Microsoft Azure

Azure Machine Learning

WHAT THE CLOUD PROVIDES



Data storage



Compute allocation



Model training



Deployment

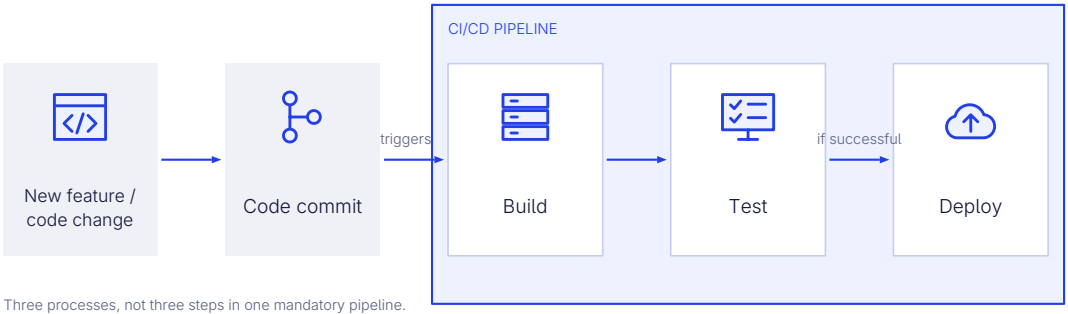


Monitoring

The major providers offer equivalent services.

Continuous Integration and Continuous Delivery (CI/CD)

Work on your application continuously and deploy new changes to it efficiently.



Three processes, not three steps in one mandatory pipeline.

CI · INTEGRATION

Run tests and code-quality checks on every revision.

CD · DELIVERY

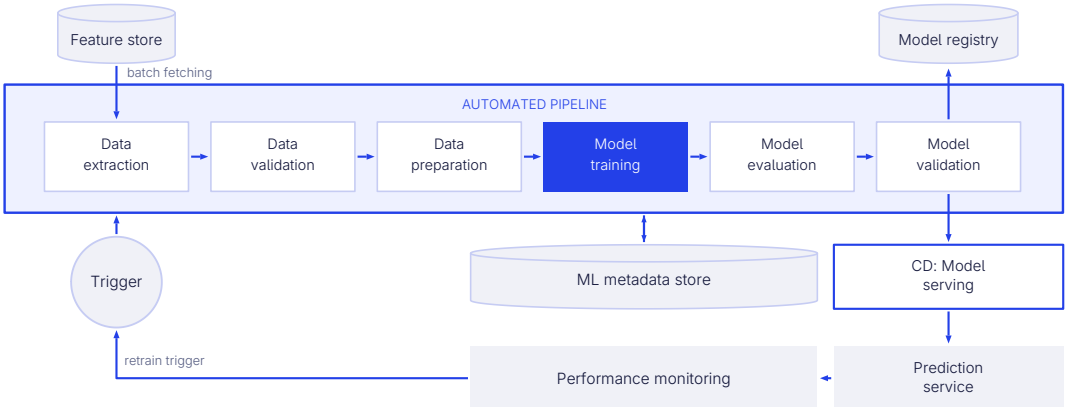
Prepare a verified package; promotion is controlled.

CT · TRAINING

Repeat training with data validation and evaluation.

ML pipeline

Orchestrates components to prepare data, train, evaluate and deploy ML models.



A completed training job is not a quality pass: evaluate and validate before promotion.

Adapted from Google Cloud Architecture Center: MLOps: continuous delivery and automation pipelines in machine learning.

Monitoring

Ensuring that models in production are performing well.



RESOURCE LEVEL

Performance and usage of resources

- How much is it used by users?
- Are CPU, RAM, network and disk as expected?
- What are the cloud costs?
- Are requests processed at the expected rate?
- What is the uptime? Contracts may depend on it.



PERFORMANCE LEVEL

Accuracy of the model over time

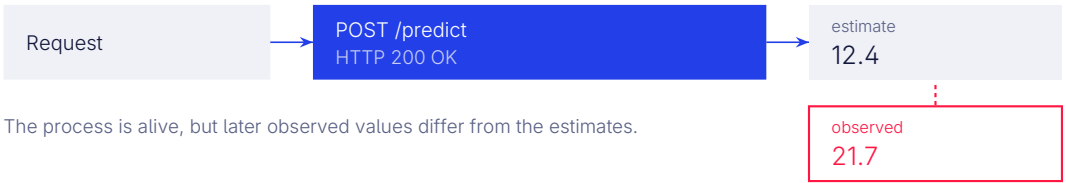
- Still accurate predictions on the new data coming in?
- Is the data distribution changing?
- Is the target variable changing?
- Are the concepts around the model changing?

A healthy process can still return poor predictions. Drift is a warning signal, not proof of accuracy loss; accuracy needs observed outcomes, which often arrive late.

A running API can still give poor estimates

An illustrative scenario, not a measured incident.

OBSERVATION



The process is alive, but later observed values differ from the estimates.

HYPOTHESES

- inputs changed
- a wrong artifact was loaded
- training used a wrong split

QUESTION

What can we verify?

A response status alone cannot identify the cause or prove model quality.

Evaluation hygiene (data splits, leakage) is covered in Lecture 2.

Trace evidence before choosing a repair

Continuing the illustrative scenario; no diagnosis is assumed.

1. **Inspect the request and observed outcome**

   Check feature names, order and values; compare predictions with available labels.

2. **Identify what actually ran**

   Match the loaded artifact to its run, training commit, data identity and configuration.

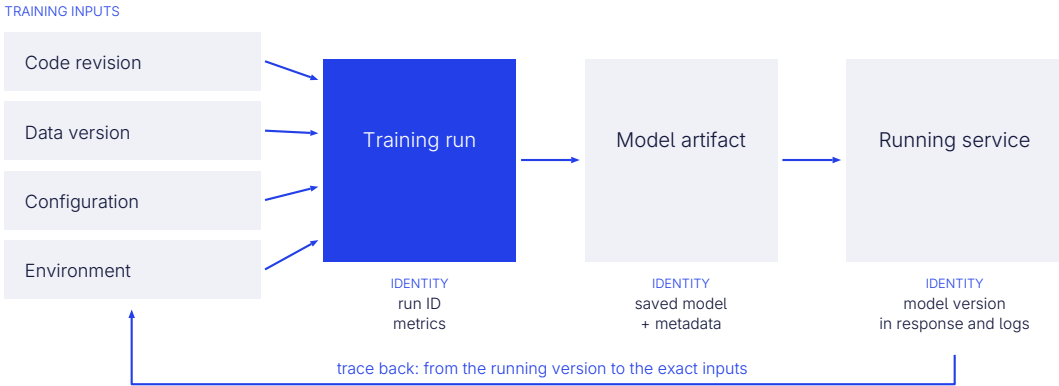
3. **Choose a cause-specific action and recheck**

   Repair an input fault, restore an artifact or evaluate a new candidate.

   Retraining is not automatic.

Trace a prediction back to its sources

Keep identities linked so another person can rerun or investigate.



Recovery must be verified

Restore a known-good service, then verify its behavior.



Recovery is done when behavior is verified, not when the service restarts.

Key concept: Trustworthy AI

The EU framework combines three foundations with seven key requirements.

THREE FOUNDATIONS

FOUNDATION 1
Lawful

+

FOUNDATION 2
Ethical

+

FOUNDATION 3
Robust

→

OUTCOME
Trustworthy AI

SEVEN KEY REQUIREMENTS

1
Human agency and oversight

2
Technical robustness and safety

3
Privacy and data governance

4
Transparency

5
Diversity, non-discrimination and fairness

6
Societal and environmental well-being

7
Accountability

The guidelines focus on the ethical and robust foundations.

Source: AI HLEG, European Commission, Ethics Guidelines for Trustworthy AI, 2019.

Teams adopt different MLOps maturity levels

Maturity levels describe capabilities; they are not grades or universal requirements.

Level	Highlights	Technology
LEVEL 0 No MLOps	• Hard to manage the full ML lifecycle • Disparate teams; painful releases • Black boxes: little feedback after deployment	• Manual training, builds and deployments • Manual testing of model and application • No centralized tracking of performance
LEVEL 1 DevOps but no MLOps	• Releases less painful than No MLOps • Limited feedback on production performance • Hard to trace and reproduce results	• Automated builds • Automated tests for application code
LEVEL 2 Automated training	• Training environment managed and traceable • Easy to reproduce a model • Releases manual, but low friction	• Automated model training • Centralized tracking of training performance • Model management
LEVEL 3 Automated deployment	• Releases low friction and automatic • Full traceability from deployment to data • Whole environment managed: dev > test > prod	• Integrated A/B testing of model performance • Automated tests for all code • Centralized tracking of training performance
LEVEL 4 Full MLOps	• Full system automated and easily monitored • Automated feedback collection and retraining • Close to zero downtime	• Automated model training and testing • Verbose, centralized metrics from the model

Source: Microsoft Azure Architecture Center, MLOps maturity model.

SECTION 4

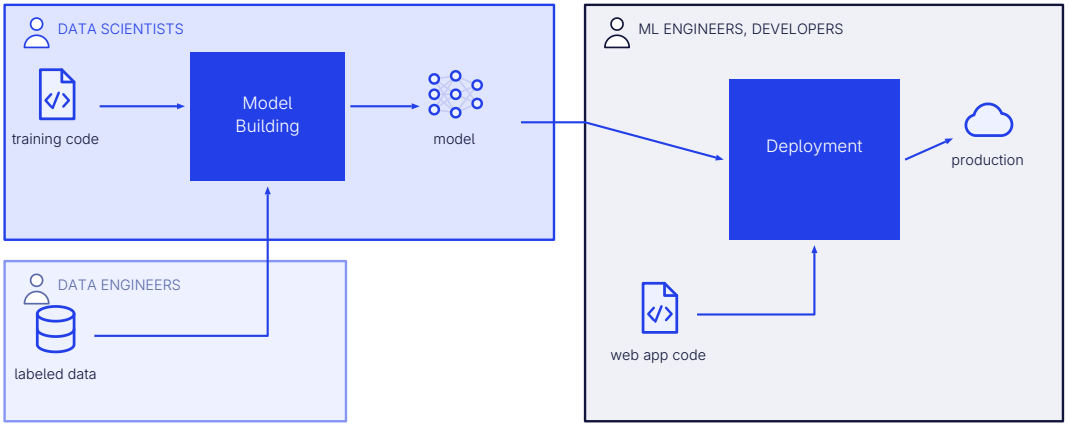
Roles & organization



ML projects: who does what, which skills matter and what teams struggle with.

Roles around an ML system implementation

Who builds which part: data, model, deployment.

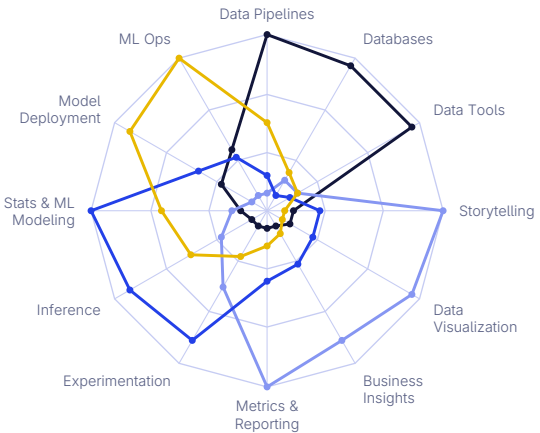


Adapted from Martin Fowler, Continuous Delivery for Machine Learning (martinfowler.com/articles/cd4ml.html).

Roles need different skill sets

Each role has a distinct profile; ML engineers sit between data science and engineering.

- **Data engineers**
  pipelines, databases, tooling
- **Data analysts**
  storytelling, visualization, insights
- **Data scientists**
  statistics, modeling, experimentation
- **ML engineers**
  deployment, MLOps, modeling



Illustrative values redrawn from Maria Vechtomova, LinkedIn post on data & AI roles; not measured data.

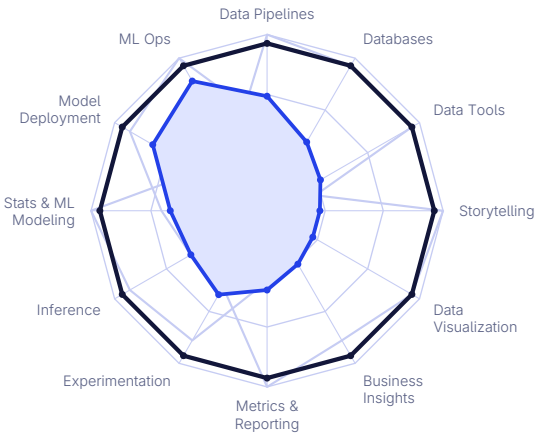
In practice, role boundaries are blurry

Job offers rarely match one textbook role.

Company X
ML engineer job offer:
modeling plus deployment
and MLOps.

Company Y
ML engineer job offer:
nearly every skill at once,
a unicorn profile.

Textbook roles (background)



Illustrative profiles after Maria Vechtomova, LinkedIn post on data & AI roles.

Most demanded skills for MLOps engineers

An analysis of 310 MLOps job offers in Q4 2023: the three most requested skills.

1



Docker

2



Python

3

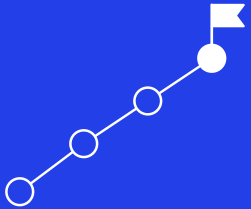


Cloud

Source: Marvelous MLOps (marvelousmlops.substack.com). Logos: Docker, Python Software Foundation, Google Material Icons.

SECTION 5

Project phases & challenges



From proof of concept to maintenance, and what makes the AI journey hard.

Build your solution in stages

From a quick proof of concept to a maintained product; durations are indicative.

PHASE 1

Proof of Concept

Use easily available data to show the model or solution can work.

Low effort: prove feasibility and value.

Iterate fast.

PHASE 2

Minimum Viable Product

Just enough features for a small set of users to start using it.

Gather feedback and make sure it is designed in an optimal way.

PHASE 3

Productionization and scaling

Build the infrastructure to deploy and let users use it.

Gradual roll-out to more users and markets.

Better models, more users, new markets.

PHASE 4

Maintenance

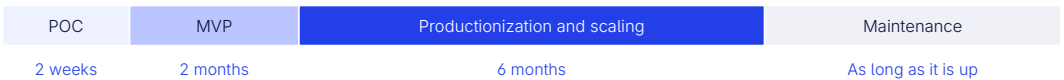
Keep the solution up and running.

Monitor resources and performance.

Update packages and dependencies.

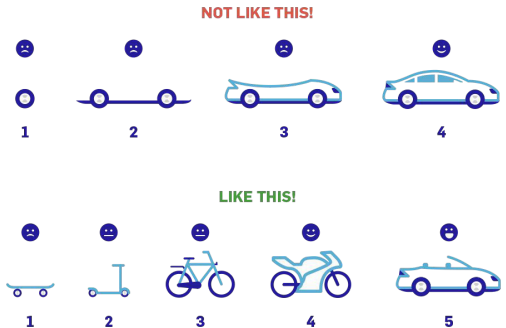
Security and up-time.

INDICATIVE TIMELINE



Grow the MVP in usable increments

Each release should already serve a user need; do not ship half of the final product.

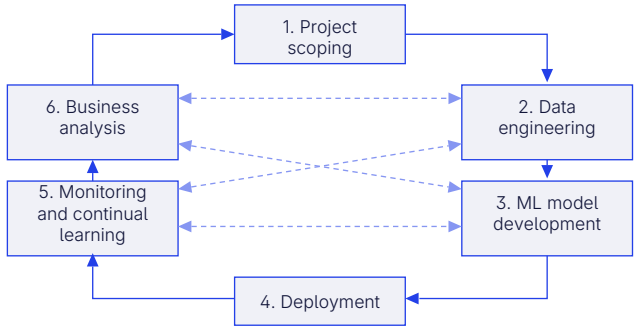


The lifecycle contains feedback

Scope, data, model, deployment and operation inform one another.

ITERATIVE WORK

A change in user needs or available inputs can send the team back to scope and evaluation.

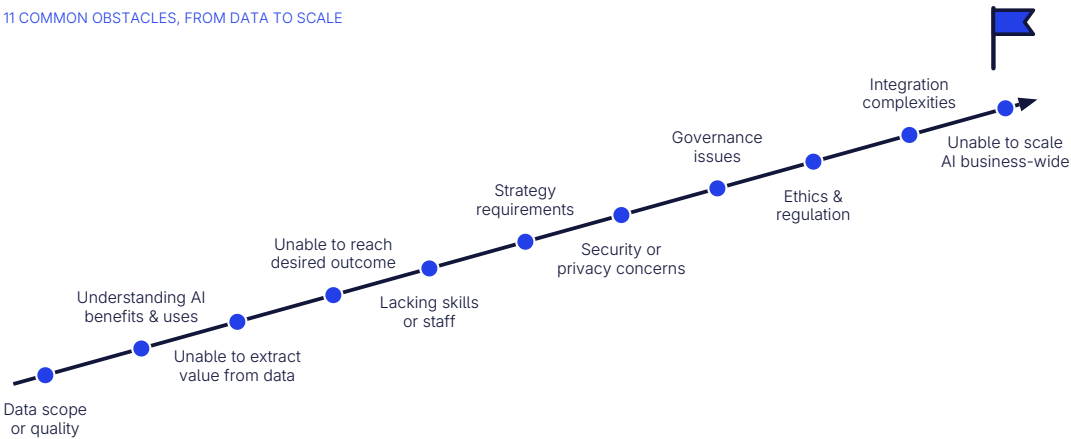


Adapted from Chip Huyen, Designing Machine Learning Systems (O'Reilly, 2022), Figure 2-2.

AI journey challenges

AI enables strategic priorities, but it brings its own challenges.

11 COMMON OBSTACLES, FROM DATA TO SCALE



SECTION 6

Real-life example: an MLOps platform



How a large organization runs ML at scale: the LinkedIn Pro-ML platform.

LinkedIn integrates many ML applications

Viral spam content detection: a boosted tree algorithm on post, member and engagement features.

Detecting spam content...



A user posts on LinkedIn at time T1.

...using a boosted tree algorithm on these features:

Post features

- Content type (image, text, video)
- Polarity and member reports

Member features

- Connection count
- Followers count
- Profile age
- Connection diversity

Engagement features

- Reactions
- Reposts
- Comments

Source: LinkedIn Engineering, Viral spam content detection at LinkedIn (linkedin.com/blog/engineering/trust-and-safety).

LinkedIn integrates many ML applications

Personalized LinkedIn news feed: a boosted tree algorithm selects content for each member.

Select personalized content for users...



...using a boosted tree algorithm on these features:

Identity

Who are you? Where do you work? What are your skills?
Who are you connected with?

Content

How many times was the update viewed or liked?
What is it about, how old is it, in which language?
Which companies, people or topics are mentioned?

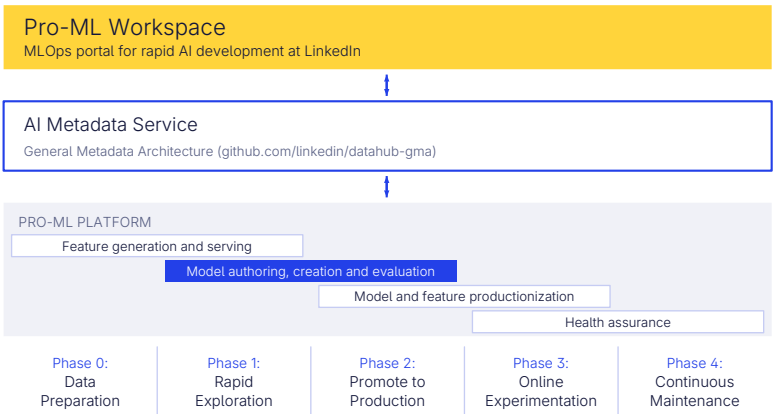
Behavior

What have you liked and shared in the past? Who do you interact with most often? Where do you spend the most time in your feed?

Source: LinkedIn Engineering, A look behind the AI that powers LinkedIn's feed (linkedin.com/blog/engineering/feed).

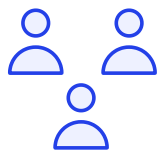
LinkedIn's Pro-ML platform

Productive Machine Learning: a one-stop MLOps portal for teams of data scientists.





Teams of data scientists

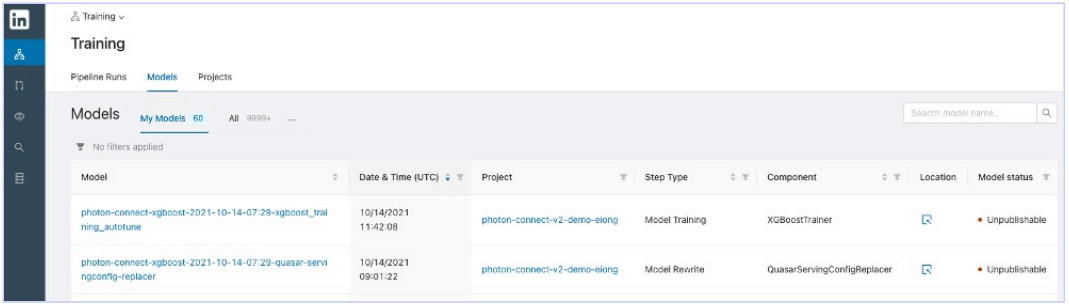


Adapted from LinkedIn Engineering, One-stop MLOps portal at LinkedIn (linkedin.com/blog/engineering/machine-learning).

LinkedIn Pro-ML platform

Step: model authoring, creation and evaluation.

Model tracking and experimentation platform. Similar to MLflow or Weights & Biases.



Training view: every model with its date, project, step type, component and status.

Source: LinkedIn Engineering, One-stop MLOps portal at LinkedIn (linkedin.com/blog/engineering/machine-learning).

LinkedIn Pro-ML platform

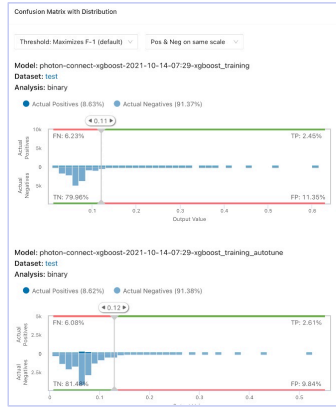
Step: model authoring, creation and evaluation: comparing candidate models.

Binary metrics:
AUROC and AUPR per model.

ROC and PR curves:
runs compared on one chart.

Confusion matrix with distribution:
pick a decision threshold
and see TP, FP, TN, FN.

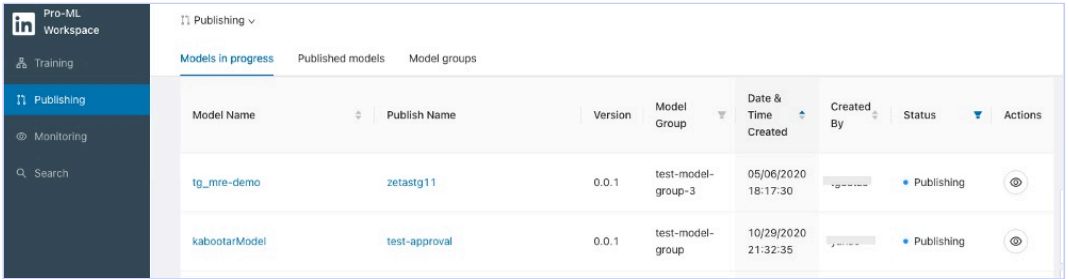




Source: LinkedIn Engineering, One-stop MLOps portal at LinkedIn (linkedin.com/blog/engineering/machine-learning).

LinkedIn Pro-ML platform

Step: model productionization.



Workflows to publish or deprecate models.

Publishing view: models in progress, published models and model groups, with version and status.

Source: LinkedIn Engineering, One-stop MLOps portal at LinkedIn (linkedin.com/blog/engineering/machine-learning).

LinkedIn Pro-ML platform

Step: health assurance, or “health insurance” for models, i.e. monitoring.

Health insurance for models

Service monitoring: calls to the model and latency over time.

Drift monitoring: a separate tab alongside the overview.



SECTION 7

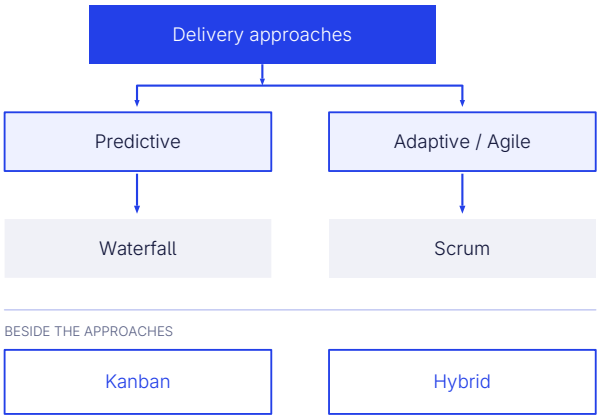
Agile ways of working



Delivery approaches, Scrum and Kanban: how teams plan and deliver work.

Software delivery approaches and frameworks

Approaches describe how work is planned; Kanban is a flow strategy that can complement them.



WATERFALL
Sequential, predictive approach

AGILE
Values and principles for adaptive software development

SCRUM
Lightweight framework for complex problems

KANBAN
Strategy for optimizing flow through a process; can complement other approaches

HYBRID
Combination of predictive and agile practices

Sources: PMI & Agile Alliance, Agile Practice Guide; Agile Manifesto (2001); Scrum Guide (2020); Kanban Guide (2025).

The Scrum team has three accountabilities

One small, cross-functional, self-managing team creates a valuable Increment each Sprint.



PRODUCT VALUE

Product Owner

Maximizes the value of the product. Manages the Product Backlog: creates and communicates its items.



SCRUM AND TEAM EFFECTIVENESS

Scrum Master

Establishes Scrum and helps the team become effective: coaches the team and removes impediments.



USABLE INCREMENT

Developers

Create a usable Increment each Sprint. Any needed skills, e.g. data scientists, ML and data engineers, DevOps, front-end.

Typically ten people or fewer: one Product Owner, one Scrum Master, the Developers.

Source: Schwaber & Sutherland, The Scrum Guide, 2020.

A sprint turns backlog items into a working increment

Each sprint lasts 1–4 weeks and repeats the same events.



SPRINT EVENTS

Sprint planning

Before the sprint, plan and move tasks from the product backlog to the sprint backlog.

Refinement

Turn each ticket into a concrete plan: approach, dependencies, definition of done.

Daily stand-up

Fixed, short questions:
– What did you do yesterday?
– What will you do today?
– Anything blocking you?

Sprint review

Present the sprint's work to the team and gather feedback: transparency and celebration.

Retrospective

Feedback on what went well and what to improve in the next sprint.

Source: Schwaber & Sutherland, The Scrum Guide, 2020.

Kanban optimizes the flow of work

Kanban literally means “visual signal”; it began in lean manufacturing at Toyota.



WHAT IT IS

- Strategy for optimizing flow through a process
- Matches work in progress (WIP) to team capacity: just in time
- Few rules; can complement other approaches

ADVANTAGES

- Planning flexibility: pick the next item; the backlog can change
- Short cycle times
- Fewer bottlenecks
- Visual metrics and transparency

Source: Kanban Guide, 2025.

SECTION 8

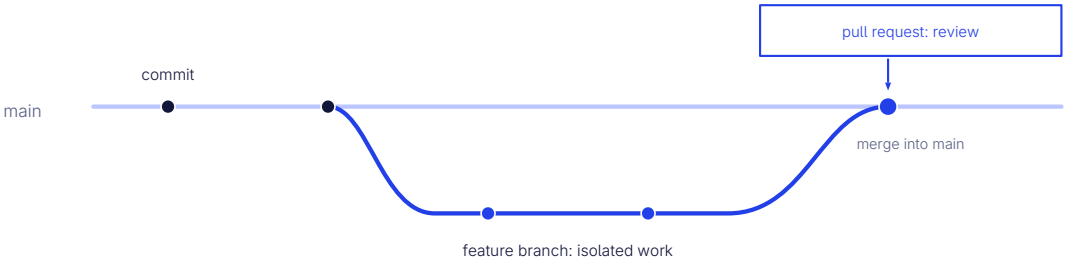
Team workflow with Gitflow



Commit, branch and pull request, then Gitflow branch roles, checkpoints and releases.

Commit, branch and PR solve different problems

Git records changes; GitHub adds collaboration and remote automation. Prefer short-lived task branches.



COMMIT

A recorded snapshot. It identifies a revision, not a test result.

BRANCH

A line of development that isolates one small change from teammates.

PULL REQUEST

Proposes a merge and links the diff, checks, feedback and response.

Gitflow separates integration from release

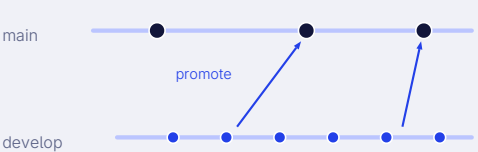
Trunk-based development integrates frequently into main; Gitflow adds a long-lived develop branch.

TRUNK-BASED



Small branches merge into main often; main is the trunk.

GITFLOW



Integrate on develop; promote accepted work to main.

In short: Gitflow keeps daily integration apart from reviewed releases.

Every branch has a clear responsibility

Keep main and develop long-lived; create task branches from the current develop.

main

STABLE · LONG-LIVED

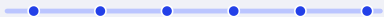


long-lived

Reviewed checkpoints and releases; no direct task commits.

develop

INTEGRATION · LONG-LIVED



long-lived

Combine accepted task PRs; keep the checks passing.

feature/*

TEMPORARY

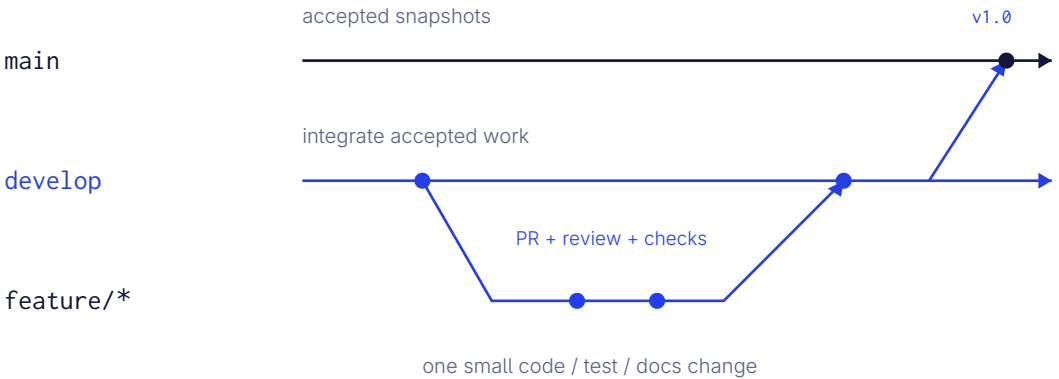


deleted after merge

One bounded change: code, tests or docs. Merge into develop.

Small task PRs feed a checked checkpoint

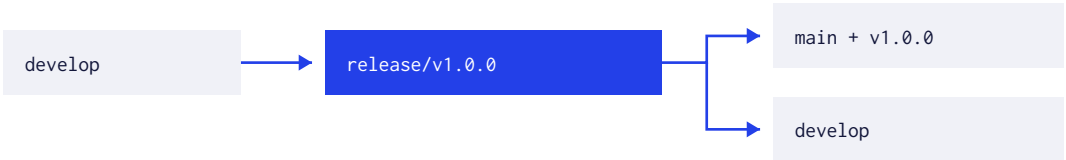
Checkpoint route: feature/* → develop → main. Final release uses release/*.



Release and hotfix changes must return

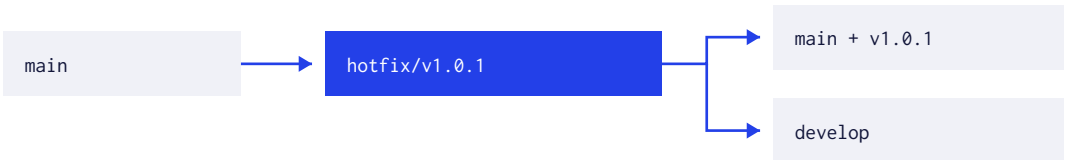
Temporary branches carry fixes to main and back to develop; do not lose the repair.

RELEASE / planned version: freeze scope, fixes only



Review and check both merge destinations; tag the accepted main commit.

HOTFIX / urgent repair of a released version



Agree names and handovers, not just branches

Record one shared convention in CONTRIBUTING.md; these patterns are examples.

Branch names

lowercase kebab-case; avoid personal or vague names

SYNTAX `<type>/<task-slug>`

e.g. `feature/l1-team-slug` `release/v1.0.0` `hotfix/v1.0.1`

Commit messages

one change, summarized

SYNTAX `<type>(<scope>): <summary>`

e.g. `fix(team): reject blank names`

Pull request descriptions

the reason and the evidence

SYNTAX `Task: <link> Why: <reason> Evidence: <tests/results> Risk: <risks>`

e.g. `Task: #12 Why: blank names crash Evidence: 3 tests pass Risk: none`

People and merge rules: one task owner, another reviewer, a named merger

Resolve feedback and current checks first; agree the merge method and the help route.

SECTION 9

Tests, review and CI



Behavior tests, meaningful review and evidence tied to one revision.

A test states a behavior you can inspect

Arrange–Act–Assert: one meaningful case, one clear failure message.

1 ARRANGE

Build the input

The data the case needs.

```
raw = -3.2
```

2 ACT

Call the code once

Run the behavior under test.

```
actual = clip_prediction(raw)
```

3 ASSERT

Check the outcome

Compare actual with expected.

```
assert actual == 0.0
```

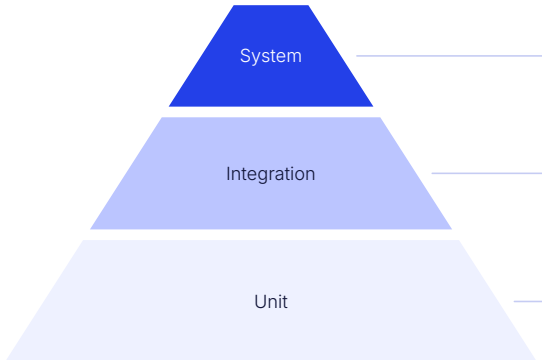
THE TEST NAME STATES THE BEHAVIOR

```
def test_negative_is_clipped():
```

If it fails, the name tells you which behavior broke.

Different tests protect different boundaries

Start with a unit contract; extend as the system grows.



System and acceptance: the usable service
Valid and invalid calls, end to end;
explain the evidence against requirements.

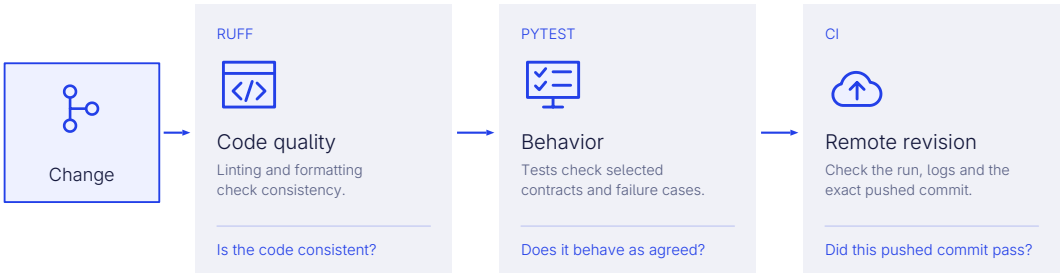
Integration: components agree
Data, split and training together,
or the API and model loading.

Unit: one function’s behavior
Selected cases and failure inputs;
a pass covers only those cases.

Lower layers: many fast, narrow checks. Higher layers: fewer, slower, broader checks.

Checks answer different questions

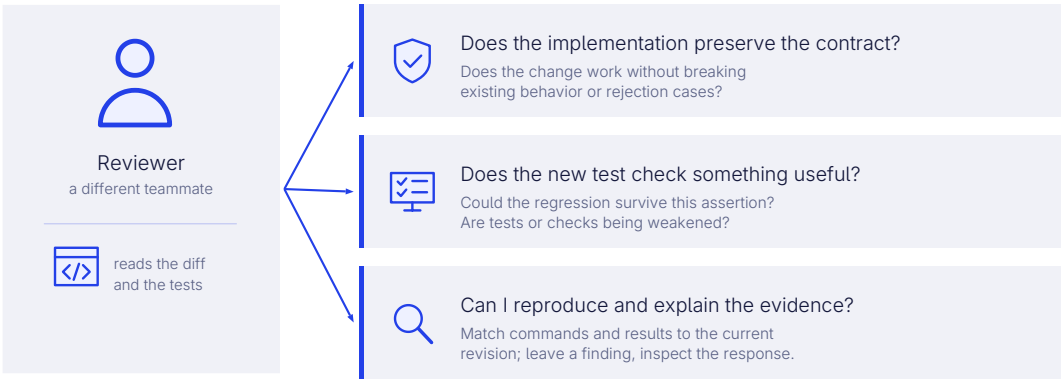
Read each actual outcome; a command alone is not evidence that it ran.



Each check answers one question; none replaces the others.

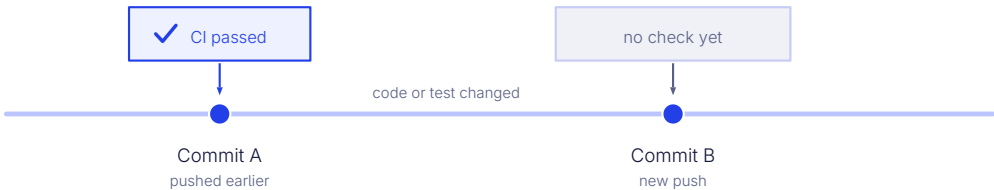
Review the behavior, not just the green badge

A different teammate should ask concrete questions about the diff.



An old green check does not validate a new commit

Local checks, remote CI and review have distinct evidence.



The green check belongs to Commit A, not to Commit B.

1 **Match status to revision**

Inspect the pushed commit, check names, conclusion and job logs.

2 **Recheck after changes**

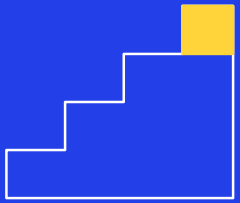
Update PR evidence after feedback; follow the team's merge agreement.

3 **Name blockers honestly**

Missing lock or CI is not a pass; record the error and next action.

SECTION 10

LLMOps and takeaway



A short introduction to LLMOps and the one principle to take away.

LLMOps adapts MLOps to LLM applications

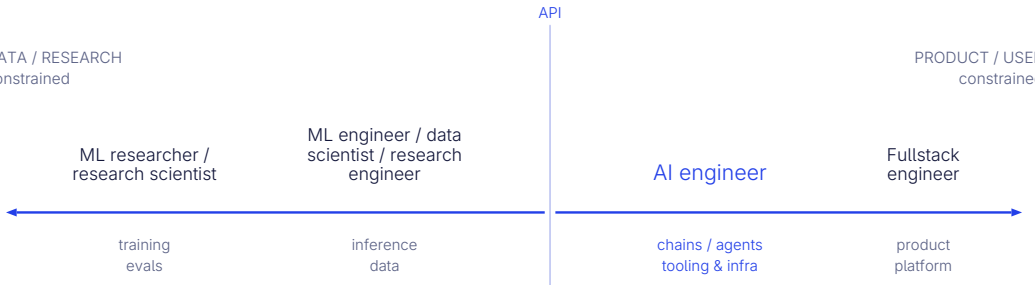
Same goal: run models reliably. Value and tooling around GenAI move fast, so each stage needs its own practices.

MLOps		LLMOps
• Prepare data for model development: cleaning, feature engineering • Source and label data	DATA MANAGEMENT	• New data stack: vector databases, embedding models, document parsing • Data access and structure matter for retrieval-augmented generation (RAG)
• Experiments alter model parameters and architecture, producing a new model • Evaluate on hold-out sets with precise metrics	EXPERIMENTS & DEVELOPMENT	• Prompt engineering and agentic workflows • Model choice, with the option to swap models • Evaluation is ambiguous and challenging
• Automated deployment of code and models across environments • Versioning and tracking of models • A/B experiments	DEPLOYMENT	• Tracing of LLM agent logs • Versioning and tracking of prompts • Guardrails and defensive mechanisms • User testing and validation before release
• Logging and monitoring of data drift and model performance	MONITORING & LOGGING	• Monitoring for bias, hallucination, other risks • LLM-as-a-judge • Latency and cost optimization

LLMs introduced a new role: the AI engineer

AI engineering builds GenAI and LLM applications on pre-trained models, accessed through APIs or open source.

AI engineering excludes custom model training but adds its own engineering practices:
prompts, chains and agents, retrieval, tooling and infrastructure around a model someone else trained.



Why now: GenAI value is rising and its tools change quickly; proper infrastructure keeps teams productive and releases safe.

Lecture 1 in summary

One idea per section.

1. **What is MLOps?**
   ML + Operations: running ML reliably in production.
2. **Why MLOps?**
   A model is not a system: it needs engineering.
3. **Key concepts of MLOps**
   Data to monitoring as one loop; traceable, recoverable.
4. **Roles & organization**
   Roles share ownership; real profiles are blurry.
5. **Project phases & challenges**
   PoC, MVP, production, maintenance; grow in increments.
6. **Real-life example**
   LinkedIn Pro-ML: author, publish and monitor models.
7. **Agile ways of working**
   Scrum and Kanban: short cycles, visible work, feedback.
8. **Team workflow with Gitflow**
   Task branches and PRs keep main and develop stable.
9. **Tests, review and CI**
   Evidence belongs to the exact revision it checked.
10. **LLMOps**
    Same discipline; new artifacts such as prompts.

One principle: a useful model needs a verifiable system.